

CS336 作业 1（基础）：构建 Transformer 语言模型

版本：26.0.3

CS336 课程组

2026 年春季

译者说明：本文是根据课程公开讲义制作的非官方中文翻译，旨在帮助中文读者学习。若译文与英文原文存在差异，请以仓库中的 `cs336_assignment1_basics.pdf` 为准。代码、API 名称、题目标识、数学符号及参考文献名称原则上保留原文。

1 作业概览

在本次作业中，你将从零开始构建训练标准 Transformer 语言模型（language model, LM）所需的全部组件，并训练若干模型。

你将实现的内容

1. 字节对编码（byte-pair encoding, BPE）分词器（第 2 节）
2. Transformer 语言模型（第 3 节）
3. 交叉熵损失函数与 AdamW 优化器（第 4 节）
4. 训练循环，并支持序列化和加载模型与优化器状态（第 5 节）

你将运行的内容

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 使用训练好的分词器对数据集编码，将其转换为整数 ID 序列。
3. 在 TinyStories 数据集上训练 Transformer 语言模型。
4. 使用训练好的 Transformer 语言模型生成样本并评估困惑度（perplexity）。
5. 在 OpenWebText 上训练模型，并将得到的困惑度提交到排行榜。

可以使用的内容

我们希望你从零开始构建每个组件。特别地，除下列内容外，你不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如 `Module`、`ModuleList`、`Sequential` 等）¹

¹完整列表见 <https://pytorch.org/docs/stable/nn.html#containers>。

- `torch.optim.Optimizer` 基类

你可以使用 PyTorch 中的其他定义。如果你想使用某个函数或类，但不确定是否允许，请在 Slack 上询问。如果拿不准，请考虑使用它是否会破坏本作业“从零实现”的初衷。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分，这会让人更难深入投入并真正学会课程内容。

允许使用 AI 工具回答高层概念问题，或提供函数签名、库 API 等底层编程文档。但不允许使用 AI 工具实现作业的任何部分，其中既包括编程智能体（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它遵守仓库中提供的 AGENTS.md。使用聊天机器人时，也应在提示词中包含该文件的内容。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全（例如 Cursor Tab、GitHub Copilot；普通的非 AI 自动补全，例如补全函数名，则完全没有问题）。往届学生特别提到，关闭 AI 自动补全让他们更容易深入学习课程材料。

完整的 AI 政策请参阅原讲义中的相应链接。

代码结构

作业代码和本讲义均发布在 GitHub：

<https://github.com/stanford-cs336/assignment1-basics>

请使用 Git 克隆该仓库。如有更新，课程组会通知你，以便你通过 `git pull` 获取最新版本。

1. `cs336_basics/*`：你将在这里编写代码。注意，这里没有现成代码，你可以完全从零开始。
2. `adapters.py`：其中定义了你的代码必须提供的一组功能。对于每项功能（例如 `scaled dot-product attention`），只需调用你自己的代码来完成相应适配函数（例如 `run_scaled_dot_product_attention`）。注意：你对 `adapters.py` 的改动不应包含任何实质性逻辑；它只是胶水代码。
3. `test_*.py`：其中包含你必须通过的全部测试（例如 `test_scaled_dot_product_attention`），这些测试会调用 `adapters.py` 中定义的接口。不要编辑测试文件。

提交方式

运行 `make_submission.sh` 生成提交用的 ZIP 文件。如果你有不希望打包进提交文件的大型数据文件或检查点，请务必将它们加入脚本的排除列表。

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请使用排版工具整理答案。
- `code.zip`：包含你编写的全部代码。

如需提交排行榜成绩，请向以下仓库提交 Pull Request：

<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

详细提交说明请参阅排行榜仓库中的 `README.md`。

数据集获取方式

本作业使用两个经过预处理的数据集：TinyStories [R. Eldan et al., 2023] 和 OpenWebText [A. Gokaslan et al., 2019]。二者都是单个大型纯文本文件。

如果你正在正式选修本课程，请参阅计算资源指南中的数据下载说明。如果你是在校外自行学习，可以使用本仓库 `README.md` 中的命令下载这些文件。

低资源提示：说明

在整门课程的作业讲义中，我们会给出一些建议，帮助你在 GPU 资源较少或完全没有 GPU 的情况下完成作业。例如，我们有时会建议缩小数据集或模型规模，或者说明如何在 Mac 集成 GPU 或 CPU 上进行训练。你会在类似这样的提示框中看到这些“低资源提示”。即使你是能够使用课程服务器的 Stanford 在校学生，这些提示也可以帮助你加快迭代并节省时间，因此我们建议阅读它们。**低资源提示：在 Apple Silicon 或 CPU 上完成作业 1**

使用课程组的参考实现，在配备 36 GB 内存的 Apple M4 Max 上，我们可以在不到 5 分钟内使用 Metal GPU (MPS) 训练出能够生成相当流畅文本的语言模型；使用 CPU 则大约需要 30 分钟。如果这些术语对你而言还很陌生，不必担心。你只需要知道：只要拥有一台配置较新的笔记本电脑，并且实现正确、高效，就能够训练一个小型语言模型，让它生成具有一定流畅度的简单儿童故事。

在作业后文中，我们会说明使用 CPU 或 MPS 时需要做哪些调整。

2 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级字节对编码 (byte-level byte-pair encoding, BPE) 分词器 [R. Sennrich et al., 2016; C. Wang et al., 2019]。具体来说，我们会将任意 (Unicode) 字符串表示为字节序列，并在该字节序列上训练 BPE 分词器。随后，我们将使用这个分词器把文本 (字符串) 编码成 token (整数序列)，供语言建模使用。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射到整数码点 (code point)。截至 2025 年 9 月发布的 Unicode 17.0，该标准在 172 种文字系统中定义了 159,801 个字符。例如，字符 `s` 的码点是 115 (通常写作 `U+0073`，其中 `U+` 是约定前缀，`0073` 是 115 的十六进制表示)，字符“牛”的码点是 29275。在 Python 中，可以使用 `ord()` 将单个 Unicode 字符转换为整数表示；`chr()` 则把整数码点转换为包含相应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题 (unicode1): 理解 Unicode (1 分)

1. `chr(0)` 返回哪个 Unicode 字符? **提交内容:** 一句话回答。
2. 该字符的字符串表示 (`__repr__()`) 与打印出来的表示有何区别? **提交内容:** 一句话回答。
3. 当该字符出现在文本中时会发生什么? 可以在 Python 解释器中尝试以下内容, 看看结果是否符合你的预期:

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容: 一句话回答。

2.2 Unicode 编码

Unicode 标准定义了从字符到码点 (整数) 的映射, 但直接在 Unicode 码点上训练分词器并不实际, 因为词表会大得难以承受 (约 15 万项), 而且非常稀疏 (许多字符极少出现)。因此, 我们将使用 Unicode 编码, 把 Unicode 字符转换成字节序列。Unicode 标准本身定义了 UTF-8、UTF-16 和 UTF-32 三种编码, 其中 UTF-8 是互联网的主流编码, 超过 98% 的网页使用它。

在 Python 中, 可以使用 `encode()` 将 Unicode 字符串编码为 UTF-8。若要访问 Python bytes 对象底层的字节值, 可以对其进行迭代 (例如调用 `list()`)。最后, 可以使用 `decode()` 将 UTF-8 字节串解码为 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值 (0 到 255 之间的整数)。
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129, 161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符!
>>> print(len(test_string))
```

13

```
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换成字节序列（例如使用 UTF-8），本质上是将一个码点序列（每项是 21 位整数，共有 159,801 个有效值）转换成字节值序列（每项是 0 到 255 之间的整数）。长度为 256 的字节词表更易于处理。使用字节级分词时，我们无需担心词表外（out-of-vocabulary）token，因为任何输入文本都可以表示成 0 到 255 之间的整数序列。

问题 (unicode2): Unicode 编码 (3 分)

1. 与 UTF-16 或 UTF-32 相比，为什么更适合在 UTF-8 编码的字节上训练分词器？比较不同输入字符串采用这些编码时的输出可能会有所帮助。**提交内容：**一到两句话回答。
2. 考虑下面这个错误的函数。它原本想把 UTF-8 字节串解码成 Unicode 字符串。这个函数为什么不正确？请给出一个会产生错误结果的输入字节串。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容：给出一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串，并用一句话解释该函数为什么不正确。3. 给出一个不能解码为任何 Unicode 字符的双字节序列。**提交内容：**给出示例并用一句话解释。

2.3 子词分词

字节级分词能够缓解词级分词器的词表外问题，但把文本拆成字节会产生极长的输入序列。这会拖慢模型训练：一个包含 10 个单词的句子在词级语言模型中可能只有 10 个 token，而在字符级模型中可能有 50 个甚至更多 token，具体取决于单词长度。处理这些较长的序列会让模型每一步都需要更多计算。此外，由于更长的输入序列会在数据中形成长期依赖，对字节序列进行语言建模本身也更困难。

子词分词位于词级分词与字节级分词之间。字节级分词器的词表有 256 项（字节值为 0 到 255）。子词分词器用更大的词表换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 经常出现在原始训练文本中，就可以在词表中为它分配一个条目，把原来的 3-token 序列缩短为单个 token。

应当如何选择要加入词表的子词单元？R. Sennrich 等人 [3] 提出使用字节对编码（BPE; P. Gage [5]）。这是一种压缩算法，会反复把最常出现的字节对替换（“合并”）为一个尚未使用的新索引。该算法通过向

词表加入子词 token 来尽可能压缩输入序列：如果某个单词在输入文本中出现得足够频繁，它最终就会表示为一个子词单元。

使用 BPE 构建词表的子词分词器通常称为 BPE 分词器。本作业将实现字节级 BPE 分词器，其词表项是字节或合并后的字节序列。这样既能避免词表外问题，又能让输入序列保持在可管理的长度。构建 BPE 分词器词表的过程称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程由三个主要步骤组成。

词表初始化

分词器词表是从字节串 token 到整数 ID 的一一映射。因为我们训练的是字节级 BPE 分词器，所以初始词表就是全部字节的集合。字节共有 256 个可能值，因此初始词表大小为 256。

预分词

有了词表之后，原则上可以统计文本中相邻字节的出现次数，再从最频繁的字节对开始合并。但这种做法计算成本很高，因为每执行一次合并都必须完整扫描一遍语料库。此外，直接在整个语料库中合并字节可能产生仅标点不同的 token（例如 `dog!` 与 `dog.`）。尽管它们很可能具有很高的语义相似度，但会得到完全不同的 token ID。

为避免这一问题，我们先对语料库进行预分词（pre-tokenization）。可以把它理解为一次粗粒度的分词，帮助我们统计字符对的出现次数。例如，单词 `text` 可能作为预 `token` 出现 10 次。统计相邻字符 `t` 和 `e` 时，我们知道 `text` 中二者相邻，于是可以直接把计数增加 10，而不必再次遍历语料库。由于我们训练的是字节级 BPE 模型，每个预 `token` 都表示为 UTF-8 字节序列。

R. Sennrich 等人 [3] 最初的 BPE 实现只是按空白字符进行预分词，即 `s.split(" ")`。基于 SentencePiece 的分词器中仍然可以见到这种方法，例如 Llama 1 和 Llama 2 的分词器。

大多数现代分词器采用基于正则表达式的预分词器，这一做法来自 GPT-2 [A. Radford et al., 2019]。我们将使用原始正则表达式的一个更整洁的版本，取自 <https://github.com/openai/tiktoken/pull/234/files>:

```
>>> PAT = r"''('?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+''"
```

可以交互式地用该预分词器拆分一些文本，以便理解它的行为：

```
>>> # 需要安装 `regex` 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '- ', 'tokenize']
```

不过，在实际代码中应使用 `re.finditer`，这样在构建“预 token 到计数”的映射时，不需要把所有预分词结果同时存入内存。

计算 BPE 合并

现在，我们已经把输入文本转换成预 token，并将每个预 token 表示成 UTF-8 字节序列，接下来就可以计算 BPE 合并，也就是训练 BPE 分词器。

从高层看，BPE 算法反复统计所有字节对，找出频率最高的一对（“A”，“B”）。随后，把这对字节的每一次出现都合并，即替换为新 token “AB”。该合并 token 会被加入词表。因此，BPE 训练后的最终词表大小等于初始词表大小（本作业中为 256）加上训练期间执行的 BPE 合并次数。为提高训练效率，我们不考虑跨越预 token 边界的字节对。²

计算合并时，如果多个字节对频率相同，应选择字典序更大的字节对，以确定性的方式打破平局。例如，如果（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）的频率都最高，应合并（“BA”，“A”）：

```
>>> max([( "A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token

有些字符串（例如 <|endoftext|>）通常用于编码元数据（例如文档边界）。编码文本时，我们往往希望把某些字符串当作“特殊 token”，永远不把它们拆成多个 token，而是始终保留为单个 token。例如，序列结束字符串 <|endoftext|> 应始终保留为单个 token（即单个整数 ID），这样语言模型生成时才能知道何时停止。必须把这些特殊 token 加入词表，使其具有固定的对应 token ID。

R. Sennrich 等人 [3] 的算法 1 给出了一种低效的 BPE 分词器训练实现，基本遵循上面概述的步骤。作为第一个练习，可以先实现并测试该函数，以检查自己是否理解了算法。

示例 (bpe_example): BPE 训练示例

下面是一个改编自 R. Sennrich 等人 [3] 的示意性示例。设语料库包含以下文本：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词表中还有一个特殊 token：<|endoftext|>。

词表

首先使用特殊 token <|endoftext|> 和 256 个字节值初始化词表。

预分词

为了简化示例并聚焦合并过程，这里假设预分词只按空白切分。完成预分词与计数后，得到以下频率表：

²R. Sennrich 等人 [3] 最初的 BPE 形式规定加入一个词尾 token。训练字节级 BPE 模型时，我们不添加词尾 token，因为所有字节（包括空白与标点）都已包含在模型词表中。既然显式表示了空格和标点，学到的 BPE 合并自然会反映这些单词边界。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

可以方便地将它表示成 `dict[tuple[bytes, ...], int]`, 例如 `{(l,o,w): 5, ...}`。注意, 在 Python 中, 即使单个字节也是一个 `bytes` 对象。Python 没有表示单个字节的 `byte` 类型, 就像它没有表示单个字符的 `char` 类型一样。

合并

首先查看每一对连续字节, 并对包含它们的单词频率求和:

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
```

字节对 ('e', 's') 与 ('s', 't') 频率相同, 因此选择字典序更大的 ('s', 't')。合并后, 预 token 变为:

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

第二轮中, (e, st) 是最常见的字节对, 计数为 9。合并后得到:

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}
```

继续执行, 最终得到的合并序列为:

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e',  
'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']
```

如果只执行 6 次合并, 则合并列表为:

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
```

此时词表元素为:

```
[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]
```

使用这个词表和合并集合, 单词 `newest` 会被分成 `[ne, west]`。

2.5 BPE 分词器训练实验

接下来在 TinyStories 数据集上训练一个字节级 BPE 分词器。数据集的查找和下载说明见第 1 节。开始之前, 建议先查看 TinyStories 数据集, 了解其中包含的内容。

并行化预分词

预分词步骤会成为主要瓶颈之一。可以使用 Python 内置的 `multiprocessing` 库并行化代码，从而加速预分词。具体而言，我们建议在并行预分词实现中，把语料库切成多个块，并确保块边界位于特殊 token 的开头。你可以原样使用以下链接中的起始代码来确定块边界，再把工作分配给各个进程：

预分词起始代码³

这种切分始终有效，因为我们从不希望在文档边界两侧执行合并。在本作业中，你始终可以采用这种切分方式，不必担心“收到一份完全不含 `<|endoftext|>` 的超大语料库”这一边界情况。

预分词前移除特殊 token

在使用正则表达式模式运行预分词（通过 `re.finditer`）之前，应从语料库中移除全部特殊 token；如果采用并行实现，则从每个块中移除。务必按特殊 token 分割文本，确保不会跨越它们分隔的文本执行合并。

例如，如果语料库或块的形式是 `[Doc 1]<|endoftext|>[Doc 2]`，应按特殊 token `<|endoftext|>` 切分，并分别对 `[Doc 1]` 与 `[Doc 2]` 进行预分词，从而避免跨文档边界合并。换言之，特殊 token 在训练期间定义了严格的分段边界，但自身不应计入合并统计。

可以使用 `re.split` 完成此操作，以 `"|".join(special_tokens)` 作为分隔符；由于特殊 token 中可能出现 `|`，需要谨慎使用 `re.escape`。测试 `test_train_bpe_special_tokens` 会检查这一点。

优化合并步骤

上面示例中的朴素 BPE 训练实现很慢，因为每执行一次合并，它都会遍历全部字节对，以找出频率最高的一对。但每次合并之后，只有与被合并字节对重叠的那些字节对计数会发生变化。因此，可以为全部字节对的计数建立索引并增量更新，而不是每次显式遍历所有字节对来重新统计频率，从而加速 BPE 训练。

这种缓存方法可以带来显著加速。不过需要注意，BPE 训练的合并部分无法在 Python 中并行化。

低资源提示：性能分析

应使用 `cProfile` 或 `py-spy` 等性能分析工具找出实现中的瓶颈，并把精力集中在优化这些部分。**低资源提示：“缩小规模”**

不要一开始就使用完整 TinyStories 数据集训练分词器。建议先在一小部分数据，也就是“调试数据集”上训练。例如，可以改用 TinyStories 验证集，其中只有 2.2 万篇文档，而不是 212 万篇。

这体现了一种通用策略：只要可能，就缩小数据集、模型大小等规模，以加快开发。选择调试数据集大小或超参数配置时需要仔细权衡：它应足够大，使瓶颈与完整配置相同，这样所做的优化才能推广；同时又不能大到每次运行都耗时过长。

³https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336-basics/pretokenization_example.py

问题 (train_bpe): 训练 BPE 分词器 (15 分)

提交内容: 编写一个函数, 给定输入文本文件的路径, 训练一个字节级 BPE 分词器。该训练函数至少应处理以下输入参数:

输入

- `input_path: str`: BPE 分词器训练数据文本文件的路径。
- `vocab_size: int`: 一个正整数, 定义最终词表的最大大小, 其中包括初始字节词表、合并产生的词表项以及全部特殊 token。
- `special_tokens: list[str]`: 要加入词表的字符串列表。训练时, 把它们当作严格边界, 禁止跨越其范围执行合并, 但在计算合并统计量时不计入这些字符串。

训练函数应返回生成的词表与合并列表:

输出

- `vocab: dict[int, bytes]`: 分词器词表, 从 `int` (词表中的 token ID) 到 `bytes` (token 的字节) 的映射。
- `merges: list[tuple[bytes, bytes]]`: 训练产生的 BPE 合并列表。每项是字节二元组 (`<token1>`, `<token2>`), 表示把 `<token1>` 与 `<token2>` 合并。列表应按合并创建顺序排列。

要使用课程组提供的测试检查 BPE 训练函数, 首先需要实现测试适配器 `adapters.run_train_bpe`, 然后运行:

```
uv run pytest tests/test_train_bpe.py
```

你的实现应能通过全部测试。作为可选项 (可能需要投入大量时间), 可以使用系统语言实现训练方法的关键部分, 例如 C++ (可考虑 `cppyy` 或 `nanobind`) 或 Rust (使用 `PyO3`)。采用这种方式时, 应注意哪些操作需要复制 Python 内存、哪些可以直接读取; 还要提供构建说明, 或确保仅凭 `pyproject.toml` 即可构建。

另请注意, 大多数正则表达式引擎对 GPT-2 正则表达式的支持不好, 并且运行速度会过慢。课程组已经验证 `Oniguruma` 速度尚可且支持负向前瞻, 不过 Python 的 `regex` 包甚至可能更快。

问题 (train_bpe_tinystories): 在 TinyStories 上训练 BPE (2 分)

1. 在 TinyStories 数据集上训练一个字节级 BPE 分词器, 最大词表大小为 10,000。确保把 TinyStories 的特殊 token `<|endoftext|>` 加入词表。将生成的词表与合并列表序列化到磁盘, 以供进一步检查。训练花费了多少时间和内存? 词表中最长的 token 是什么? 它是否合理?

资源要求: 不使用 GPU 时不超过 30 分钟, 内存不超过 30 GB。

提示: 如果在预分词期间使用多进程, 并利用以下两个事实, BPE 训练时间应能控制在 2 分钟以内:

1. 数据文件中的 `<|endoftext|>` token 用于分隔文档。
2. `<|endoftext|>` token 在应用 BPE 合并之前作为特殊情况处理。

提交内容：一到两句话回答。2. 对代码进行性能分析。分词器训练过程中的哪一部分最耗时？**提交内容：**一到两句话回答。

接下来，我们尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。与之前一样，建议先浏览数据集，以便更好地了解其内容。

问题 (train_bpe_expts_owt)：在 OpenWebText 上训练 BPE (2 分)

1. 在 OpenWebText 数据集上训练一个字节级 BPE 分词器，最大词表大小为 32,000。将生成的词表与合并列表序列化到磁盘，以供进一步检查。词表中最长的 token 是什么？它是否合理？

资源要求：不使用 GPU 时不超过 12 小时，内存不超过 100 GB。**提交内容：**一到两句话回答。2. 对比在 TinyStories 与 OpenWebText 上训练得到的分词器。**提交内容：**一到两句话回答。

2.6 BPE 分词器：编码与解码

在上一部分中，我们实现了一个在输入文本上训练 BPE 分词器的函数，从而得到分词器词表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器：它加载给定的词表与合并列表，并使用它们在文本和 token ID 之间进行编码与解码。

2.6.1 编码文本

BPE 文本编码过程与 BPE 词表的训练过程相似，主要包括以下步骤。

第 1 步：预分词。与 BPE 训练时相同，首先对序列进行预分词，并把每个预 token 表示成 UTF-8 字节序列。我们会在每个预 token 内部把这些字节合并成词表元素；各预 token 独立处理，不允许跨越预 token 边界合并。

第 2 步：应用合并。取得 BPE 训练期间创建的词表元素合并序列，再按照它们的创建顺序依次应用到预 token。

示例 (bpe_encoding)：BPE 编码示例

假设输入字符串为 'the cat ate'，词表为：

```
{
  0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't',
  6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'
}
```

学到的合并为：

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]
```

首先, 预分词器把字符串拆成 ['the', ' cat', ' ate']。然后逐个查看预 token 并应用 BPE 合并。

第一个预 token 'the' 最初表示为 [b't', b'h', b'e']。查看合并列表, 第一项可应用的合并是 (b't', b'h'), 它把预 token 转换成 [b'th', b'e']。随后再次查看合并列表, 下一项可应用的合并是 (b'th', b'e'), 得到 [b'the']。此时已经没有更多合并可应用, 因为整个预 token 已合并为单个 token, 因此处理结束。对应的整数序列是 [9]。

对其余预 token 重复这一过程: ' cat' 应用 BPE 合并后表示为 [b' c', b'a', b't'], 对应整数序列 [7, 1, 5]; 最后的预 token ' ate' 表示为 [b' at', b'e'], 对应整数序列 [10, 3]。因此, 输入字符串的最终编码结果是 [9, 7, 1, 5, 10, 3]。

特殊 token

编码文本时, 分词器应能正确处理用户定义的特殊 token; 这些 token 在构造分词器时提供。

内存方面的考虑

假设要对一个无法全部装入内存的大型文本文件进行分词。为了高效处理该文件或其他数据流, 需要把它切成大小可管理的块, 再依次处理每一块。这样, 内存复杂度就是常数, 而不是随文本大小线性增长。

切块时必须确保 token 不会跨越块边界, 否则所得分词结果会与“把整个序列一次性载入内存”的朴素方法不同。

2.6.2 解码文本

要把整数 token ID 序列解码回原始文本, 只需在词表中查找每个 ID 对应的条目 (字节序列), 把这些字节序列连接起来, 再将结果解码为 Unicode 字符串。

注意, 输入 ID 不保证能够映射为有效的 Unicode 字符串, 因为用户可以输入任意整数 ID 序列。如果输入 token ID 没有产生有效的 Unicode 字符串, 应使用官方 Unicode 替换字符 U+FFFD 替换格式错误的字节。⁴ bytes.decode 的 errors 参数控制如何处理 Unicode 解码错误; 设为 errors='replace' 会自动用替换字符代替错误数据。

问题 (tokenizer): 实现分词器 (15 分)

提交内容: 实现一个 Tokenizer 类。给定词表与合并列表, 该类可以把文本编码为整数 ID, 也可以把整数 ID 解码为文本。分词器还应支持用户提供的特殊 token; 如果某个特殊 token 尚未存在于词表中, 就把它追加到词表。

推荐使用以下接口:

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、合并列表以及可选的特殊 token 列表构造分词器。该函数应接受:

⁴Unicode 替换字符的更多信息见 [https://en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

- vocab: dict[int, bytes]
- merges: list[tuple[bytes, bytes]]
- special_tokens: list[str] | None = None

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化后的词表和合并列表（格式与你的 BPE 训练代码输出相同）以及可选的特殊 token 列表构造并返回 `Tokenizer`。它还应接受：

- vocab_filepath: str
- merges_filepath: str
- special_tokens: list[str] | None = None

```
def encode(self, text: str) -> list[int]
```

把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定字符串可迭代对象（例如 Python 文件句柄），返回一个惰性产生 token ID 的生成器。为了以内存高效的方式对无法直接载入内存的大型文件分词，必须实现这一接口。

```
def decode(self, ids: list[int]) -> str
```

把 token ID 序列解码为文本。

要使用课程组提供的测试检查 `Tokenizer`，首先实现测试适配器 `adapters.get_tokenizer`，然后运行：

```
uv run pytest tests/test_tokenizer.py
```

你的实现应能通过全部测试。

2.7 实验

问题 (`tokenizer_experiments`)：分词器实验（4 分）

1. 分别从 `TinyStories` 和 `OpenWebText` 中抽取 10 篇文档。使用先前训练的 `TinyStories` 与 `OpenWebText` 分词器（词表大小分别为 10K 和 32K），将样本文档编码为整数 ID。两个分词器各自的压缩率（字节数/token 数）是多少？**提交内容**：一到两句话回答。
2. 如果使用 `TinyStories` 分词器对 `OpenWebText` 样本分词，会发生什么？比较压缩率，和/或从定性角度描述结果。**提交内容**：一到两句话回答。

- 估算分词器的吞吐量，例如每秒处理的字节数。对 Pile 数据集（825 GB 文本）完成分词需要多长时间？**提交内容：**一到两句话回答。
- 使用 TinyStories 和 OpenWebText 分词器，分别把相应的训练集与开发集编码成整数 token ID 序列。稍后我们将使用这些数据训练语言模型。建议把 token ID 序列化为数据类型为 uint16 的 NumPy 数组。为什么 uint16 是合适的选择？**提交内容：**一到两句话回答。

3 Transformer 语言模型架构

语言模型的输入是一批整数 token ID 序列，即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`；输出是词表上的一批归一化概率分布，即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch Tensor。对于每个输入 token，预测分布描述下一个 token。

训练语言模型时，我们使用这些下一个 token 的预测，计算真实下一个 token 与预测结果之间的交叉熵损失。推理生成文本时，取最后一个时间步（即序列最后一项）的下一个 token 预测分布，用它生成序列中的下一个 token，例如选择概率最高的 token 或从分布中采样；随后把生成的 token 加到输入序列末尾，并重复此过程。

本部分将从零构建这个 Transformer 语言模型。我们先从模型的高层描述开始，再逐步介绍各个组件。

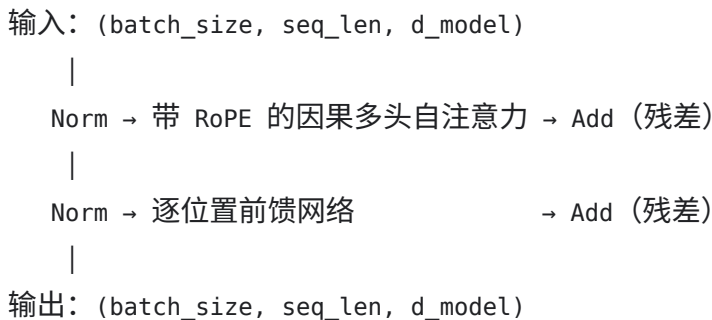
3.1 Transformer 语言模型

给定 token ID 序列，Transformer 语言模型首先使用输入嵌入（input embedding）把 token ID 转换为稠密向量；然后让嵌入后的 token 通过 `num_layers` 个 Transformer block；最后应用一个学习得到的线性投影，即“输出嵌入”（output embedding）或“LM head”，生成对下一个 token 的 logits 预测。

图 1：Transformer 语言模型概览



图 2: Pre-norm Transformer block



Token 嵌入

第一步中, Transformer 把一批 token ID 序列嵌入为包含 token 身份信息的向量序列 (原图 1 中的红色块)。

更具体地说, 给定 token ID 序列, Transformer 语言模型使用 token 嵌入层生成一个向量序列。嵌入层接收形状为 (batch_size, sequence_length) 的整数 Tensor, 输出形状为 (batch_size, sequence_length, d_model) 的向量序列。

Pre-norm Transformer block

完成嵌入后, 激活值会依次通过若干结构相同的神经网络层。标准的仅解码器 (decoder-only) Transformer 语言模型包含 num_layers 个相同的层, 它们通常称为 Transformer “block”。每个 Transformer block 接收形状为 (batch_size, sequence_length, d_model) 的输入, 并返回相同形状的输出。

每个 block 通过自注意力在序列中聚合信息, 再通过前馈层进行非线性变换。经过 num_layers 个 Transformer block 后, 我们把最终激活值转换成词表上的分布。

本作业将实现 “pre-norm” Transformer block (详见第 3.4 节)。为确保输出具有适当尺度, 还需要在最后一个 Transformer block 之后使用层归一化。完成归一化后, 再使用标准的、可学习的线性变换, 把 Transformer block 的输出转换为下一个 token 的预测 logits (例如参见 A. Radford 等人 [7] 的公式 2)。

3.2 补充说明: 批处理、Einsum 与高效计算

在整个 Transformer 中, 我们会对许多类似 batch 的输入应用相同计算。例如:

- **batch 中的元素:** 对 batch 中的每个元素应用相同的 Transformer 前向计算。
- **序列长度维度:** RMSNorm 和前馈网络等 “逐位置” 操作, 会以相同方式处理序列中的每个位置。
- **注意力头:** 多头注意力操作会在多个注意力头上批量执行注意力计算。

我们需要一种易于使用的方式来执行这些操作, 使其既能充分利用 GPU, 又便于阅读和理解。许多 PyTorch 操作可以接受位于 Tensor 开头的额外 “类 batch” 维度, 并在这些维度上高效地重复或广播操作。

例如, 假设要执行一个逐位置的批量操作。现有 “数据 Tensor” D , 形状为 (batch_size, sequence_length, d_model); 希望它与形状为 (d_model, d_model) 的矩阵 A 执行批量向量-矩阵乘

法。此时, `D @ A` 会执行批量矩阵乘法, 这是 PyTorch 的高效原语, 其中 `(batch_size, sequence_length)` 两个维度作为 `batch` 维度处理。

因此, 最好假设函数可能收到额外的类 `batch` 维度, 并始终把这些维度放在 PyTorch `shape` 的开头。为了把 `Tensor` 整理成能够如此批处理的形状, 可能需要经过多次 `view`、`reshape` 和 `transpose`。这会有些繁琐, 代码执行的操作和 `Tensor shape` 也常常变得难以理解。

一种更易用的方式是在 `torch.einsum` 中采用 `einsum` 记法, 或者使用与框架无关的 `einops`、`einx` 等库。两个关键操作是:

- `einsum`: 对输入 `Tensor` 的任意维度执行张量缩并 (tensor contraction)。
- `rearrange`: 对任意维度重新排序、拼接和拆分。

事实证明, 机器学习中的几乎所有操作都是维度整理与张量缩并的某种组合, 间或再加上通常为逐元素的非线性函数。因此, 使用 `einsum` 记法可以让大量代码更易读、更灵活。

本课程强烈建议学习并使用 `einsum` 记法。此前没有接触过 `einsum` 的学生应使用 `einops` (请先阅读其文档); 已经熟悉 `einops` 的学生可以学习更通用的 `einx`。⁵ 课程组提供的环境已经安装了这两个包。

以下示例补充说明 `einsum` 记法的用法; 阅读之前, 应先阅读 `einops` 文档。

示例 (`einstein_example1`): 使用 `einops.einsum` 进行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基本实现
Y = D @ A.T
# 很难看出输入和输出的 shape 及其含义。
# D 和 A 可以具有哪些 shape? 其中是否会出现意外行为?

## Einsum 具有自我说明性, 而且更稳健
#
#           D           A       ->       Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者使用批量版本: D 可以具有任意前导维度, 但 A 的维度受约束。
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (`einstein_example2`): 使用 `einops.rearrange` 进行广播操作

假设有一批图像, 希望根据某个缩放因子, 为每幅图像生成 10 个变暗版本:

⁵`einops` 得到了广泛支持, 而 `einx` 尚未经过同等程度的实战检验。如果遇到 `einx` 的限制或 bug, 可以随时改用 `einops`, 并搭配一些普通 PyTorch 操作。

```

images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 改变形状并相乘
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一次完成:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)

```

示例 (einstein_example3): 使用 einops.rearrange 混合像素

假设有一批形状为 (batch, height, width, channel) 的图像, 想对图像的所有像素执行线性变换, 但每个通道应独立变换。线性变换由形状为 (height * width, height * width) 的矩阵 B 表示。

```

channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32 * 32, 32 * 32)

## 重新排列图像 Tensor, 以便在全部像素之间进行混合
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)

## 使用 einops:
height = width = 32
## rearrange 取代了冗长的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(

```

```

    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

## 或者更大胆一点: 使用 einx.dot 一次完成
## (einx 中与 einops.einsum 对应的操作)
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)

```

第一种实现可以通过在前后添加注释来说明输入与输出 shape, 但这样既笨重又容易出错。使用 `einsum` 记法时, 实现本身就是文档。

`Einsum` 记法能够处理任意输入批处理维度, 而且具有自我说明这一重要优点。在代码中, 输入和输出 `Tensor` 的相关 shape 会清晰得多。

对于其余 `Tensor`, 可以考虑使用 `Tensor` 类型提示, 例如 `jaxtyping` 库 (它并非只能用于 JAX)。作业 2 会进一步讨论 `einsum` 记法对性能的影响; 目前只需知道, 它几乎总比替代写法更合适。

3.2.1 数学记法与内存顺序

许多机器学习论文在记法中使用行向量, 这种表示与 NumPy 和 PyTorch 默认采用的行优先 (row-major) 内存顺序配合得很好。使用行向量时, 线性变换写作:

$$y = xW^T, \quad (1)$$

其中, 按行优先存储的 $W \in \mathbb{R}^{d_{out} \times d_{in}}$, 行向量 $x \in \mathbb{R}^{1 \times d_{in}}$ 。这样只需增大 x 最外层的维度就能批处理输入, 即可用矩阵输入 $X \in \mathbb{R}^{batch \times d_{in}}$ 替代向量输入 x 。

在线性代数中, 更常见的是使用列向量; 此时线性变换写作:

$$y = Wx, \quad (2)$$

其中, $W \in \mathbb{R}^{d_{out} \times d_{in}}$ 仍按行优先存储, 列向量 $x \in \mathbb{R}^{d_{in}}$ 。若要在这种设定下批处理输入, x 的 batch

维度必须放在最后，因此需要把 x 替换为矩阵 $\tilde{X} \in \mathbb{R}^{d_{in} \times batch}$ 。

本作业的数学记法大多采用列向量，因为数学文献通常遵循这一惯例。需要牢记：PyTorch 采用行优先内存顺序，因此如果想直接使用普通的矩阵乘法记法，就必须像公式 (1) 的行向量约定那样，对矩阵应用转置。如果在线性代数操作中使用 einsum，只要正确标记各个轴，这就不再是问题。

顺带一提，Matlab、Julia 和 Fortran 等其他语言或线性代数包都采用列优先内存顺序，因此 batch 维度放在最后；Python 及其相关软件包则沿用了 C 标准的行优先顺序。

3.3 基本构件：Linear 与 Embedding 模块

3.3.1 参数初始化

有效训练神经网络通常需要仔细初始化模型参数。不合适的初始化可能导致梯度消失或梯度爆炸等不良行为。Pre-norm Transformer 对初始化异常稳健，但初始化仍会显著影响训练速度与收敛情况。由于本作业篇幅已经很长，相关细节留到作业 3 再讨论；这里先给出一组近似初始化方法，在大多数情况下应当表现良好。

目前请使用：

- Linear 权重： $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{in} + d_{out}})$ ，截断到 $[-3\sigma, 3\sigma]$ 。
- Embedding： $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ ，截断到 $[-3, 3]$ 。
- RMSNorm：1。

应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 Linear 模块

线性层是 Transformer 以及一般神经网络的基本构件。首先，实现自己的 Linear 类，使其继承 `torch.nn.Module` 并执行线性变换：

$$y = Wx. \quad (3)$$

遵循大多数现代大语言模型的做法，这里不包含偏置项。

问题 (linear)：实现 Linear 模块 (1 分)

提交内容：实现一个继承 `torch.nn.Module` 并执行线性变换的 Linear 类。除不提供偏置参数或偏置项外，实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口。推荐接口如下：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

构造一个线性变换模块。该函数应接受：

- `in_features`: int: 输入的最后一个维度。

- `out_features: int`: 输出的最后一个维度。
- `device: torch.device | None = None`: 存储参数的设备。
- `dtype: torch.dtype | None = None`: 参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入应用线性变换。

务必做到:

- 继承 `nn.Module`;
- 调用父类构造函数;
- 以 W 而不是 W^T 的形式构造并存储参数, 并把它放入 `nn.Parameter`;
- 不使用 `nn.Linear` 或 `nn.functional.linear`。

初始化时, 使用上面的设定, 并通过 `torch.nn.init.trunc_normal_` 初始化权重。

要测试 `Linear` 模块, 请实现测试适配器 `adapters.run_linear`。适配器应把给定权重载入你的 `Linear` 模块, 可以使用 `Module.load_state_dict` 完成。然后运行:

```
uv run pytest -k test_linear
```

3.3.3 Embedding 模块

如前所述, Transformer 的第一层是嵌入层, 把整数 token ID 映射到维度为 `d_model` 的向量空间。我们将实现一个继承 `torch.nn.Module` 的自定义 `Embedding` 类, 因此不应使用 `nn.Embedding`。

`forward` 方法应接收形状为 `(batch_size, sequence_length)`、包含 token ID 的 `torch.LongTensor`, 用这些 ID 索引形状为 `(vocab_size, d_model)` 的嵌入矩阵, 从而为每个 token ID 选出对应的嵌入向量。

问题 (embedding): 实现 Embedding 模块 (1 分)

提交内容: 实现一个继承 `torch.nn.Module` 并执行嵌入查找的 `Embedding` 类。实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。推荐接口如下:

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造嵌入模块。该函数应接受:

- `num_embeddings: int`: 词表大小。
- `embedding_dim: int`: 嵌入向量的维度, 即 d_{model} 。

- `device: torch.device | None = None`: 存储参数的设备。
- `dtype: torch.dtype | None = None`: 参数的数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找给定 token ID 的嵌入向量。

务必做到:

- 继承 `nn.Module`;
- 调用父类构造函数;
- 把嵌入矩阵初始化为 `nn.Parameter`;
- 存储嵌入矩阵时, 把 `d_model` 作为最后一个维度;
- 不使用 `nn.Embedding` 或 `nn.functional.embedding`。

同样, 使用前述设定以及 `torch.nn.init.trunc_normal_` 初始化权重。

要测试实现, 请完成测试适配器 `adapters.run_embedding`, 然后运行:

```
uv run pytest -k test_embedding
```

3.4 Pre-Norm Transformer Block

每个 Transformer block 有两个子层: 多头自注意力机制和逐位置前馈网络 (A. Vaswani 等人 [2017], 第 3.1 节)。

在最初的 Transformer 论文中, 模型在两个子层外分别使用残差连接, 随后执行层归一化。这种架构通常称为 “post-norm” Transformer, 因为层归一化应用于子层的输出。但许多研究发现, 把层归一化从每个子层的输出移到输入, 并在最后一个 Transformer block 之后额外加入一次层归一化, 可以提高 Transformer 的训练稳定性 [T. Q. Nguyen et al., 2019; R. Xiong et al., 2020]。图 2 展示了这种 “pre-norm” Transformer block。

每个 Transformer block 子层的输出再通过残差连接加到该子层的输入上 (A. Vaswani 等人 [8], 第 5.4 节)。对 pre-norm 的一种直观理解是: 从输入嵌入到 Transformer 最终输出之间存在一条没有任何归一化的、干净的 “残差流” (residual stream), 据称这有助于梯度流动。

如今的语言模型通常采用 pre-norm Transformer, 例如 GPT-3、LLaMA、PaLM 等, 因此本作业也将实现这一变体。下面依次介绍并实现 pre-norm Transformer block 的各个组件。

3.4.1 均方根层归一化

A. Vaswani 等人 [8] 最初的 Transformer 实现使用层归一化 [J. L. Ba et al., 2016] 来归一化激活值。遵循 H. Touvron 等人 [12] 的做法, 本作业使用均方根层归一化 (root mean square layer normalization, RMSNorm; B. Zhang 等人 [13], 公式 4)。

给定激活向量 $a \in \mathbb{R}^{d_{model}}$, RMSNorm 按如下方式重新缩放每个激活值 a_i :

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

其中:

$$\text{RMS}(a) = \sqrt{\frac{1}{d_{model}} \sum_{i=1}^{d_{model}} a_i^2 + \varepsilon}.$$

g_i 是可学习的“增益” (gain) 参数, 总共有 `d_model` 个; ε 是超参数, 通常固定为 `1e-5`。

在对输入求平方前, 应先把它向上转换为 `torch.float32`, 以防溢出。整体上, `forward` 方法应具有如下结构:

```
in_dtype = x.dtype
x = x.to(torch.float32)

# 在这里编写执行 RMSNorm 的代码
...
result = ...

# 以原始 dtype 返回结果
return result.to(in_dtype)
```

问题 (rmsnorm): 均方根层归一化 (1 分)

提交内容: 将 RMSNorm 实现为 `torch.nn.Module`。推荐接口如下:

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

构造 RMSNorm 模块。该函数应接受:

- `d_model: int`: 模型的隐藏维度。
- `eps: float = 1e-5`: 用于保证数值稳定性的 epsilon 值。
- `device: torch.device | None = None`: 存储参数的设备。
- `dtype: torch.dtype | None = None`: 参数的数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

处理形状为 `(batch_size, sequence_length, d_model)` 的输入 Tensor, 并返回相同形状的 Tensor。

注意: 如上所述, 执行归一化之前, 记得把输入向上转换为 `torch.float32`, 之后再转换回原始 `dtype`。

要测试实现, 请完成测试适配器 `adapters.run_rmsnorm`, 然后运行:

```
uv run pytest -k test_rmsnorm
```

3.4.2 逐位置前馈网络

图 3：SiLU（又称 Swish）与 ReLU 激活函数的比较。原图同时绘制了恒等函数 $f(x) = x$ 、ReLU $f(x) = \max(0, x)$ 和 SiLU $f(x) = x\sigma(x)$ ；SiLU 在零点附近保持平滑，并在负半轴保留较小的非零输出。

原始 Transformer 论文（A. Vaswani 等人 [8]，第 3.3 节）中的前馈网络由两次线性变换组成，中间使用 ReLU 激活函数：

$$\text{ReLU}(x) = \max(0, x).$$

在这一原始架构中，内部前馈层的维度通常是输入维度的 4 倍。

现代语言模型与原始设计相比，往往有两项主要变化：使用另一种激活函数，并采用门控机制。具体而言，本作业将实现 Llama 3 [A. Grattafiori et al., 2024]、Qwen 2.5 [A. Yang et al., 2024] 等大语言模型采用的“SwiGLU”激活函数。它把 SiLU（通常也称为 Swish）激活与门控线性单元（Gated Linear Unit, GLU）结合起来。

此外，遵循自 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来大多数现代大语言模型的做法，我们省略线性层中有时会使用的偏置项。

SiLU 或 Swish 激活函数 [D. Hendrycks et al., 2016; S. Elfwing et al., 2017] 定义为：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}. \quad (5)$$

与 ReLU 相比，SiLU 的整体形状相似，但在零点处是平滑的。

门控线性单元最初由 Y. N. Dauphin 等人 [19] 定义为：一个经过 sigmoid 函数的线性变换，与另一个线性变换逐元素相乘：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中 $\odot$ 表示逐元素乘法。GLU 被认为能够“为梯度提供线性路径，同时保留非线性能力，从而减轻深层架构中的梯度消失问题”。

把 SiLU/Swish 与 GLU 结合起来，就得到前馈网络将使用的 SwiGLU：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2 (\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

其中：

$$x \in \mathbb{R}^{d_{\text{model}}}, \quad W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}},$$

通常取 $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ 。具体实现时，可以把它舍入到附近的 64 的倍数，以提高硬件效率。

N. Shazeer [20] 最早提出把 SiLU/Swish 激活与 GLU 结合，并通过实验表明，在语言建模任务上，SwiGLU 优于 ReLU 和不带门控的 SiLU 等基线。稍后你将比较 SwiGLU 与 SiLU。尽管我们已经提到这些组件的一些启发式解释，相关论文也提供了更多支持证据，但仍应保持实证视角。Shazeer 论文中有一句如今颇为著名的话：

“我们无法解释这些架构为何似乎有效；和其他一切一样，我们把它们的成功归因于神意眷顾。”

问题 (positionwise_feedforward)：实现逐位置前馈网络 (2 分)

提交内容：实现 SwiGLU 前馈网络，它由 SiLU 激活函数与 GLU 组成。

注意：在这个特定问题中，为保证数值稳定性，可以直接使用 `torch.sigmoid`。

实现中应把 d_{ff} 设为约 $\frac{8}{3}d_{model}$ ，同时保证内部前馈层的维度是 64 的倍数，以充分利用硬件。

要使用课程组提供的测试检查实现，需要完成测试适配器 `adapters.run_swiglu`，然后运行：

```
uv run pytest -k test_swiglu
```

3.4.3 相对位置嵌入

为了向模型注入位置信息，我们将实现旋转位置嵌入 (Rotary Position Embeddings; J. Su et al., 2021)，通常简称 RoPE。

对于 token 位置 i 上的给定 query token：

$$q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d,$$

我们应用成对旋转矩阵 R^i ，得到：

$$q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}.$$

R^i 会把每一对嵌入元素 $q_{2k-1:2k}^{(i)}$ 视为二维向量，并旋转角度：

$$\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}, \quad k \in \{1, \dots, d/2\},$$

其中 Θ 是某个常数。因此，可以把 R^i 看作一个 $d \times d$ 的分块对角矩阵，其第 k 个块为：

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix}. \quad (8)$$

完整旋转矩阵为：

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

其中每个 0 都表示 2×2 零矩阵。

尽管可以构造完整的 $d \times d$ 矩阵，一个好的实现应利用其结构性质，更高效地完成变换。我们只关心给定序列中 token 之间的相对旋转，因此各层及不同 batch 可以复用 $\cos(\theta_{i,k})$ 与 $\sin(\theta_{i,k})$ 的计算结果。

如果希望优化，可以让所有层引用同一个 RoPE 模块，并在初始化时使用：

```
self.register_buffer(persistent=False)
```

创建预计算的二维正弦和余弦值缓冲区，而不是使用 `nn.Parameter`，因为这些固定的正弦和余弦值不应被学习。

对 $q^{(i)}$ 执行的同一旋转过程也要应用于 $k^{(j)}$ ，使用对应的 R^j 旋转。注意，这一层没有任何可学习参数。

问题 (rope): 实现 RoPE (2 分)

提交内容：实现一个把 RoPE 应用于输入 Tensor 的 `RotaryPositionalEmbedding` 类。

推荐接口如下：

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

构造 RoPE 模块，并在需要时创建缓冲区：

- `theta: float`: RoPE 的 Θ 值。
- `d_k: int`: query 与 key 向量的维度。
- `max_seq_len: int`: 可能输入的最大序列长度。
- `device: torch.device | None = None`: 存储缓冲区的设备。

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

处理形状为 $(\dots, \text{seq_len}, d_k)$ 的输入 Tensor，并返回相同形状的 Tensor。应允许 x 带有任意数量的 batch 维度。假设 token 位置是形状为 $(\dots, \text{seq_len})$ 的 Tensor，指定 x 在序列维度上的 token 位置。

应使用这些 token 位置沿序列维度切片可能已经预计算的 `cos` 与 `sin` Tensor。

要测试实现，请完成 `adapters.run_rope`，并确保以下命令通过：

```
uv run pytest -k test_rope
```

3.4.4 缩放点积注意力

现在实现 A. Vaswani 等人 [8] 第 3.2.1 节所述的缩放点积注意力。作为准备, Attention 操作会使用 softmax。它把未归一化的分数向量转换成归一化分布:

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

注意, 对于很大的值, $\exp(v_i)$ 可能变成 inf , 随后出现 $\frac{\text{inf}}{\text{inf}} = \text{NaN}$ 。可以利用 softmax 对“给全部输入加上同一常数 c ”保持不变这一性质来避免该问题。为保证数值稳定性, 通常从 v 的每个元素中减去 v 的最大值, 使新的最大元素为 0。

问题 (softmax): 实现 softmax (1 分)

提交内容: 编写一个对 Tensor 应用 softmax 操作的函数。函数接收两个参数: 一个 Tensor 和一个维度 i , 并沿输入 Tensor 的第 i 个维度应用 softmax。输出 Tensor 的 shape 应与输入相同, 但其第 i 个维度现在是归一化概率分布。

为避免数值稳定性问题, 应从第 i 个维度的全部元素中减去该维度上的最大值。

要测试实现, 请完成 `adapters.run_softmax`, 并确保以下命令通过:

```
uv run pytest -k test_softmax_matches_pytorch
```

Attention 操作在数学上定义为:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$ 、 $K \in \mathbb{R}^{m \times d_k}$ 、 $V \in \mathbb{R}^{m \times d_v}$ 。 Q 、 K 、 V 都是该操作的输入, 不是可学习参数。

掩码 (masking): 有时需要遮蔽注意力操作的部分输出。掩码的形状应为 $M \in \{\text{True}, \text{False}\}^{n \times m}$; 这个布尔矩阵的第 i 行说明 query i 应关注哪些 key。

按照惯例, 一个稍显容易混淆的地方是: 位置 (i, j) 的值为 `True`, 表示 query i 会关注 key j ; 值为 `False` 则表示不会关注。换言之, 信息会沿值为 `True` 的 (i, j) 对流动。例如, 考虑条目为 `[[True, True, False]]` 的 1×3 掩码矩阵, 唯一的 query 向量只关注前两个 key。

从计算角度看, 使用掩码远比在多个子序列上分别计算注意力高效。可以在 softmax 之前的值 $QK^T/\sqrt{d_k}$ 上, 把掩码矩阵中所有 `False` 位置加上 $-\infty$ 来实现遮蔽。

问题 (scaled_dot_product_attention): 实现缩放点积注意力 (5 分)

提交内容: 实现缩放点积注意力函数。实现应处理形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_k)$ 的 key 与 query, 以及形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$ 的 value; 其中 $\dots$ 表示任意数量的其他类 batch 维度 (如果存在)。返回输出的形状应为 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$ 。关于类 batch 维度的说明见第 3.2 节。

实现还应支持用户提供的可选布尔掩码，其形状为 $(\text{seq_len}, \text{seq_len})$ 。掩码值为 `True` 的位置，其注意力概率加总后应为 1；掩码值为 `False` 的位置，其注意力概率应为零。

要使用课程组提供的测试检查实现，需要完成测试适配器：

```
adapters.run_scaled_dot_product_attention
```

以下命令使用三阶输入 Tensor 测试实现：

```
uv run pytest -k test_scaled_dot_product_attention
```

以下命令则使用四阶输入 Tensor 测试实现：

```
uv run pytest -k test_4d_scaled_dot_product_attention
```

3.4.5 因果多头自注意力

我们将实现 A. Vaswani 等人 [8] 第 3.2.2 节所述的多头自注意力。回顾其数学定义：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h), \quad (12)$$

其中：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i). \quad (13)$$

Q_i, K_i, V_i 分别是 Q, K, V 的嵌入维度上第 $i \in \{1, \dots, h\}$ 个大小为 d_k 或 d_v 的切片。这里的 `Attention` 就是第 3.4.4 节定义的缩放点积注意力。

由此可以得到多头自注意力操作：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x). \quad (14)$$

其中，可学习参数为：

$$W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}, \quad W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}, \quad W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}.$$

由于 Q, K, V 会在多头注意力操作中被切片，可以把 W_Q, W_K, W_V 看作沿输出维度为各个注意力头分别存放的权重。当实现正确后，`query`、`key` 和 `value` 三个投影总共应只需要三次矩阵乘法。⁶

因果掩码

实现必须防止模型关注序列中未来的 token。换言之，给定 token 序列 $t_1, \dots, t_n$ ，如果要为前缀 $t_1, \dots, t_i$ （其中 $i < n$ ）计算下一个 token 的预测，模型就不能访问或关注位置 $t_{i+1}, \dots, t_n$ 的 token 表示。推理生成

⁶作为扩展目标，可以尝试把 `key`、`query` 和 `value` 投影合并到单个权重矩阵中，使其只需要一次矩阵乘法。

文本时模型尚未见到这些 token，而且未来 token 会泄露真实下一个 token 的信息，让语言模型预训练目标变得毫无难度。

对于输入序列 $t_1, \dots, t_n$ ，一种朴素方法是为序列中的 n 个不同前缀分别运行一次多头自注意力，总共运行 n 次。我们改用因果注意力掩码，使 token i 能关注序列中所有满足 $j \leq i$ 的位置。可以使用 `torch.triu` 或广播后的索引比较来构造该掩码，并应利用第 3.4.4 节的缩放点积注意力实现已经支持掩码这一事实。

应用 RoPE

RoPE 应用于 query 和 key 向量，但不应用于 value 向量。此外，应把注意力头维度当作 batch 维度处理，因为在多头注意力中，每个头会独立应用注意力。这意味着每个注意力头的 query 和 key 向量都要应用完全相同的 RoPE 旋转。

问题 (multihead_self_attention): 实现因果多头自注意力 (5 分)

提交内容: 把因果多头自注意力实现为 `torch.nn.Module`。实现至少应接受以下参数：

- `d_model`: `int`: Transformer block 输入的维度。
- `num_heads`: `int`: 多头自注意力使用的头数。

遵循 A. Vaswani 等人 [8]，设：

$$d_k = d_v = \frac{d_{model}}{h}.$$

要使用课程组提供的测试检查实现，请完成测试适配器 `adapters.run_multihead_self_attention`，然后运行：

```
uv run pytest -k test_multihead_self_attention
```

3.5 完整的 Transformer 语言模型

首先组装 Transformer block；此时回看图 2 会有所帮助。一个 Transformer block 包含两个“子层”：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，先执行 RMSNorm，再执行主要操作（MHA 或 FFN），最后加入残差连接。

具体来说，Transformer block 的前半部分，也就是第一个子层，应执行以下更新，从输入 x 得到输出 y ：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

问题 (transformer_block): 实现 Transformer block (3 分)

按照第 3.4 节的描述和图 2 的示意, 实现 pre-norm Transformer block。实现至少应接受以下参数:

- `d_model`: int: Transformer block 输入的维度。
- `num_heads`: int: 多头自注意力使用的头数。
- `d_ff`: int: 逐位置前馈网络内部层的维度。

要测试实现, 请完成适配器 `adapters.run_transformer_block`, 然后运行:

```
uv run pytest -k test_transformer_block
```

提交内容: 能够通过课程组测试的 Transformer block 代码。

接下来按照图 1 的高层结构把各个 block 组合起来。按照第 3.1 节对嵌入的说明, 把嵌入结果送入 `num_layers` 个 Transformer block, 随后通过最后的层归一化与 LM head, 得到词表上的未归一化分布, 即 logits。

问题 (`transformer_lm`): 实现 Transformer 语言模型 (3 分)

现在把所有组件组合起来。按照第 3.1 节的描述和图 1 的示意, 实现 Transformer 语言模型。实现至少应接受前面提到的 Transformer block 全部构造参数, 以及下列额外参数:

- `vocab_size`: int: 词表大小, 用于确定 token 嵌入矩阵的维度。
- `context_length`: int: 最大上下文长度, 用于确定 RoPE 正弦与余弦缓冲区的维度。
- `num_layers`: int: 使用的 Transformer block 数量。

要使用课程组提供的测试检查实现, 首先完成测试适配器 `adapters.run_transformer_lm`, 然后运行:

```
uv run pytest -k test_transformer_lm
```

提交内容: 能够通过上述测试的 Transformer 语言模型模块。

资源核算

理解 Transformer 各个部分如何消耗计算量和内存非常有用。下面介绍基本的“FLOPs 核算”。Transformer 中绝大多数 FLOPs 来自矩阵乘法, 因此核心方法很简单:

1. 列出 Transformer 一次前向传播中的所有矩阵乘法。
2. 把每次矩阵乘法换算成所需 FLOPs。

第二步会用到以下规则:

规则: 给定 $A \in \mathbb{R}^{m \times n}$ 与 $B \in \mathbb{R}^{n \times p}$, 矩阵乘积 AB 需要 $2mnp$ FLOPs。

理由如下: $(AB)[i, j] = A[i, :] \cdot B[:, j]$, 这个点积需要 n 次加法与 n 次乘法, 即 $2n$ FLOPs。由于矩阵乘积 AB 包含 $m \times p$ 个元素, 总 FLOPs 就是 $(2n)(mp) = 2mnp$ 。

开始下一题之前, 建议逐一查看 Transformer block 和 Transformer 语言模型的每个组件, 列出全部矩阵乘法及各自的 FLOPs 成本。

问题 (transformer_accounting): Transformer 语言模型资源核算 (5 分)

1. 考虑一个采用本作业架构、大小与 GPT-2 XL 相当的模型, 配置如下:

```
vocab_size:      50,257
context_length:   1,024
num_layers:       48
d_model:          1,600
num_heads:        25
d_ff:             4,288 (最接近 (8/3) * 1,600 的 64 的倍数)
```

假设按此配置构造模型, 它有多少个可训练参数? 假设每个参数都用单精度浮点数表示, 仅加载这个模型需要多少内存? **提交内容:** 一到两句话回答。2. 找出这个 GPT-2 XL 形状模型完成一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs? 假设输入序列包含 context_length 个 token。 **提交内容:** 列出矩阵乘法及其说明, 并给出所需 FLOPs 总数。3. 根据上述分析, 模型的哪些部分需要最多 FLOPs? **提交内容:** 一到两句话回答。4. 对 GPT-2 small (12 层、d_model=768、12 个头)、GPT-2 medium (24 层、d_model=1024、16 个头) 和 GPT-2 large (36 层、d_model=1280、20 个头) 重复分析。随着模型增大, Transformer 语言模型的哪些部分占总 FLOPs 的比例变大或变小? **提交内容:** 对每个模型给出各组件及其 FLOPs 占一次前向传播总 FLOPs 的比例; 另外, 用一到两句话描述模型大小变化如何改变各组件的 FLOPs 占比。5. 使用 GPT-2 XL, 并把上下文长度增加到 16,384。一次前向传播的总 FLOPs 如何变化? 各模型组件的 FLOPs 相对贡献如何变化? **提交内容:** 一到两句话回答。

4 训练 Transformer 语言模型

现在, 我们已经有了预处理数据的步骤 (分词器) 和模型 (Transformer)。剩下的工作是构建支持训练的全部代码, 其中包括:

- **损失:** 定义损失函数, 即交叉熵。
- **优化器:** 定义使损失最小化的优化器, 即 AdamW。
- **训练循环:** 构建加载数据、保存检查点和管理训练所需的全部基础设施。

4.1 交叉熵损失

回顾一下：对于每个长度为 $m + 1$ 的序列 x 和每个 $i = 1, \dots, m$ ，Transformer 语言模型定义分布 $p_\theta(x_{i+1} | x_{1:i})$ 。给定由长度为 $m + 1$ 的序列组成的训练集 D ，标准交叉熵（负对数似然）损失函数定义为：

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_\theta(x_{i+1} | x_{1:i}). \quad (16)$$

注意，Transformer 的一次前向传播就会为所有 $i = 1, \dots, m$ 产生 $p_\theta(x_{i+1} | x_{1:i})$ 。

具体而言，Transformer 会为每个位置 i 计算 logits $o_i \in \mathbb{R}^{vocab_size}$ ，因此：

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{vocab_size} \exp(o_i[a])}. \quad (17)$$

这里， $o_i[k]$ 表示向量 o_i 在索引 k 处的值。交叉熵损失通常针对 logits 向量 $o_i \in \mathbb{R}^{vocab_size}$ 与目标 x_{i+1} 定义；这对应于 x_{i+1} 上的狄拉克 delta 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

与 softmax 一样，实现交叉熵损失时也必须谨慎处理数值问题。

问题 (cross_entropy)：实现交叉熵 (1 分)

提交内容：编写一个计算交叉熵损失的函数。该函数接收预测 logits o_i 和目标 x_{i+1} ，计算：

$$\ell_i = -\log \text{softmax}(o_i)[x_{i+1}].$$

函数应做到：

- 减去最大元素，以保证数值稳定性；
- 尽可能约去 log 与 exp；
- 处理任意额外的 batch 维度，并返回整个 batch 的平均值。与第 3.2 节相同，假设类 batch 维度始终位于词表大小维度之前。

实现 `adapters.run_cross_entropy`，然后运行：

```
uv run pytest -k test_cross_entropy
```

困惑度

交叉熵足以用于训练，但评估模型时还要报告困惑度。对于长度为 m 、交叉熵损失依次为 $\ell_1, \dots, \ell_m$ 的序列：

$$\text{perplexity} = \exp \left(\frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

4.2 SGD 优化器

有了损失函数之后,开始探索优化器。最简单的基于梯度的优化器是随机梯度下降(Stochastic Gradient Descent, SGD)。从随机初始化的参数 θ_0 开始, 对于每一步 $t = 0, \dots, T - 1$, 执行更新:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中, B_t 是从数据集 D 中随机抽取的一批数据; 学习率 α_t 与 batch 大小 $|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现优化器,我们将继承 PyTorch 的 `torch.optim.Optimizer` 类。`Optimizer` 子类必须实现两个方法:

```
def __init__(self, params, ...)
```

该方法初始化优化器。`params` 是待优化参数的集合; 如果用户希望对模型不同部分使用不同超参数(例如不同学习率), 也可以传入参数组。务必把 `params` 传给基类的 `__init__`, 基类会保存这些参数供 `step` 使用。根据优化器需要, 还可以接收额外参数, 例如常见的学习率, 并把它们以字典形式传给基类构造函数; 字典的键是你为这些参数选择的名称字符串。

```
def step(self)
```

该方法执行一次参数更新。在训练循环中, 它会在反向传播之后调用, 因此可以访问最后一个 batch 上的梯度。该方法应遍历每个参数 Tensor `p`, 并原地修改它们, 即设置 `p.data`。`p.data` 保存与该参数关联的 Tensor; 更新应依据 `p.grad` (如果存在), 后者表示损失对该参数的梯度 Tensor。

PyTorch 优化器 API 有一些细节, 用一个示例更容易说明。为了让示例更丰富, 我们实现一个略有变化的 SGD: 学习率在训练过程中衰减, 从初始学习率 α 开始, 步长随时间逐渐减小:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t). \quad (20)$$

这个 SGD 版本可实现为:

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
```



```

    defaults = {"lr": lr}
    super().__init__(params, defaults)

def step(self, closure: Optional[Callable] = None):
    loss = None if closure is None else closure()
    for group in self.param_groups:
        lr = group["lr"] # 获取学习率。
        for p in group["params"]:
            if p.grad is None:
                continue

            state = self.state[p] # 获取与 p 关联的状态。
            t = state.get("t", 0) # 从状态中获取迭代次数；不存在则取 0。
            grad = p.grad.data # 获取损失关于 p 的梯度。
            p.data -= lr / math.sqrt(t + 1) * grad # 原地更新权重 Tensor。
            state["t"] = t + 1 # 递增迭代次数。

    return loss

```

在 `__init__` 中，我们把参数和默认超参数传给基类构造函数；这些参数也可能分组，每组使用不同超参数。如果参数只是一个 `torch.nn.Parameter` 对象集合，基类构造函数会创建单个参数组并为其分配默认超参数。

随后，在 `step` 中依次遍历各参数组与组内参数，并应用公式 (20)。这里把迭代次数保存为与每个参数关联的状态：先读取其值，在梯度更新中使用，再更新该值。

API 规定用户可能传入可调用的 `closure`，以便在优化器更新前重新计算损失。本作业使用的优化器不需要这一功能，但为了符合 API，仍把它加入实现。

可以使用下面这个最小训练循环示例观察它如何工作：

```

weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 重置全部可学习参数的梯度。
    loss = (weights**2).mean() # 计算标量损失值。
    print(loss.cpu().item())
    loss.backward() # 执行反向传播，计算梯度。
    opt.step() # 执行一次优化器更新。

```

这就是训练循环的典型结构：每次迭代计算损失，再执行一次优化器更新。训练语言模型时，可学习

参数来自模型；在 PyTorch 中，`m.parameters()` 会给出这一参数集合。损失会在随机抽取的一个 batch 上计算，但训练循环的基本结构相同。

问题 (learning_rate_tuning): 调整学习率 (1 分)

正如后面会看到的，学习率是对训练影响最大的超参数之一。用上面的玩具示例实际观察这一点：分别把学习率改为 $1e1$ 、 $1e2$ 和 $1e3$ ，每种情况只执行 10 次训练迭代。各学习率下的损失会发生什么？它下降得更快、更慢，还是会发散，也就是在训练过程中增大？

提交内容：用一到两句话描述观察到的行为。

4.3 AdamW

现代语言模型通常不使用 SGD，而是采用更复杂的优化器。近年来使用的大多数优化器都是 Adam [D. P. Kingma et al., 2015] 的衍生版本。本作业使用 AdamW [I. Loshchilov et al., 2019]，它在近期研究中应用广泛。

AdamW 对 Adam 做了一项修改：通过加入权重衰减改善正则化，即每次迭代都把参数向 0 拉近，同时使该操作与梯度更新解耦。我们将按照 I. Loshchilov 等人 [23] 的算法 2 实现 AdamW。

AdamW 是有状态的：它会为每个参数维护一阶矩与二阶矩的运行估计。因此，AdamW 使用额外内存，以换取更好的稳定性与收敛性。除学习率 α 外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2) ，以及权重衰减率 λ 。

典型应用把 (β_1, β_2) 设为 $(0.9, 0.999)$ ；LLaMA [H. Touvron et al., 2023] 和 GPT-3 [T. B. Brown et al., 2020] 等大语言模型则经常使用 $(0.9, 0.95)$ 。算法如下，其中 ϵ 是一个很小的值，例如 10^{-8} ，用于在 v 极小时提高数值稳定性。

算法 1: AdamW 优化器

1. `init( $\theta$ )`: 初始化可学习参数。
2. $m \leftarrow 0$: 一阶矩向量的初值，shape 与 θ 相同。
3. $v \leftarrow 0$: 二阶矩向量的初值，shape 与 θ 相同。
4. 对 $t = 1, \dots, T$:
5. 抽取一批数据 B_t 。
6. $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$: 计算损失的梯度。
7. $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$: 计算第 t 次迭代调整后的 α 。
8. $\theta \leftarrow \theta - \alpha \lambda \theta$: 应用权重衰减。
9. $m \leftarrow \beta_1 m + (1 - \beta_1) g$: 更新一阶矩估计。
10. $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$: 更新二阶矩估计。

11. $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v+\epsilon}}$: 应用经矩估计调整的权重更新。
12. 结束循环。

注意, t 从 1 开始。现在实现这个优化器。

问题 (adamw): 实现 AdamW (2 分)

提交内容: 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。类的 `__init__` 应接收学习率 α , 以及 β , ϵ 和 λ 超参数。

为了帮助保存状态, `Optimizer` 基类提供字典 `self.state`, 把 `nn.Parameter` 对象映射到保存该参数所需信息的字典; 对 AdamW 而言, 其中应包括矩估计。

实现 `adapters.get_adamw_cls`, 并确保以下命令通过:

```
uv run pytest -k test_adamw
```

问题 (adamw_accounting): AdamW 训练资源核算 (2 分)

下面计算运行 AdamW 所需的内存与计算量。假设每个 Tensor 都使用 `float32`。

1. 运行 AdamW 需要多少峰值内存? 请按参数、激活值、梯度和优化器状态的内存用量拆解答案, 并使用 `batch_size` 以及模型超参数 `vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads` 表示答案。假设 $d_{ff} = \frac{8}{3}d_{model}$ 。

为简化激活值内存的计算, 只考虑以下组件:

- Transformer block
 - RMSNorm;
 - 多头自注意力子层: QKV 投影、 QK^T 矩阵乘法、softmax、value 的加权求和、输出投影;
 - 逐位置前馈网络 (SwiGLU): W_1 、 W_2 、门控分支上的 SiLU、逐元素乘法、 W_3 。
- 最后的 RMSNorm;
- 输出嵌入;
- logits 上的交叉熵。

提交内容: 分别给出参数、激活值、梯度和优化器状态的代数表达式, 并给出总量。2. 把上述答案具体代入 GPT-2 XL 形状模型, 得到只依赖 `batch_size` 的表达式。在 80 GB 内存中仍能容纳的最大 `batch size` 是多少? **提交内容:** 形如 $a \cdot \text{batch_size} + b$ 的表达式, 其中 a, b 为数值; 另给出最大 `batch size`。3. 运行一步 AdamW 需要多少 FLOPs? **提交内容:** 给出代数表达式并简要说明理由。4. 模型 FLOPs 利用率 (model FLOPs utilization, MFU) 定义为: 观测到的吞吐量 (token/秒) 相对于硬件理论峰值 FLOP 吞吐量的比率 [A. Chowdhery et al., 2022]。NVIDIA H100 GPU 对 “float32” (实际是 `TensorFloat-32`, 现实

中是“bfloat19”) 操作的理论峰值为 495 teraFLOP/s。假设能够达到 50% MFU, 在单张 H100 上用 batch size 1024 训练 GPT-2 XL 共 400K 步, 需要多长时间? 遵循 J. Kaplan 等人 [25] 和 J. Hoffmann 等人 [26], 假设反向传播的 FLOPs 是前向传播的两倍。**提交内容:** 给出训练所需小时数并简要说明理由。

4.4 学习率调度

能够让损失最快下降的学习率通常会随训练过程而变化。训练 Transformer 时, 一般使用学习率调度: 开始阶段使用较大的学习率, 以便更快更新; 随着模型训练, 再缓慢衰减到较小值。⁷ 本作业将实现训练 LLaMA [H. Touvron et al., 2023] 时采用的余弦退火调度。

调度器只是一个函数: 它接收当前步骤 t 和其他相关参数, 例如初始与最终学习率, 然后返回步骤 t 的梯度更新所应使用的学习率。最简单的调度是常数函数, 无论输入哪个 t , 都返回同一个学习率。

余弦退火学习率调度接收:

1. 当前迭代 t ;
2. 最大学习率 α_{max} ;
3. 最小 (最终) 学习率 α_{min} ;
4. 预热 (warm-up) 迭代次数 T_w ;
5. 余弦退火的最后一次迭代 T_c 。

迭代 t 的学习率定义如下。

预热: 若 $t < T_w$, 则:

$$\alpha_t = \frac{t}{T_w} \alpha_{max}.$$

余弦退火: 若 $T_w \leq t \leq T_c$, 则:

$$\alpha_t = \alpha_{min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{max} - \alpha_{min}).$$

退火后: 若 $t > T_c$, 则:

$$\alpha_t = \alpha_{min}.$$

问题 (learning_rate_schedule): 实现带预热的余弦学习率调度 (1 分)

编写一个函数, 接收 t 、 α_{max} 、 α_{min} 、 T_w 与 T_c , 并按照上面定义的调度器返回学习率 α_t 。然后实现 `adapters.get_lr_cosine_schedule`, 确保以下命令通过:

```
uv run pytest -k test_get_lr_cosine_schedule
```

⁷有时也会使用让学习率重新升高 (restart) 的调度, 以帮助越过局部极小值。

4.5 梯度裁剪

训练期间, 有时会遇到产生巨大梯度的训练样本, 从而破坏训练稳定性。实践中常用梯度裁剪 (gradient clipping) 缓解该问题: 每次反向传播后、执行优化器更新前, 对梯度范数施加上限。

给定全部参数的梯度 g , 先计算其 ℓ_2 范数 $\|g\|_2$ 。如果该范数小于最大值 M , 则保持 g 不变; 否则, 把 g 缩小 $\frac{M}{\|g\|_2 + \epsilon}$ 倍, 其中加入很小的 ϵ (例如 10^{-6}) 以保证数值稳定性。得到的新范数会略小于 M 。

问题 (gradient_clipping): 实现梯度裁剪 (1 分)

编写一个实现梯度裁剪的函数。函数接收参数列表与最大 ℓ_2 范数, 并原地修改每个参数的梯度。使用 $\epsilon = 10^{-6}$, 这也是 PyTorch 的默认值。

随后实现适配器 `adapters.run_gradient_clipping`, 并确保以下命令通过:

```
uv run pytest -k test_gradient_clipping
```

5 训练循环

现在终于可以把目前构建的主要组件组合起来: 完成分词的数据、模型与优化器。

5.1 数据加载器

完成分词的数据, 例如在 `tokenizer_experiments` 中准备的数据, 是单个 token 序列:

$$x = (x_1, \dots, x_n).$$

即使源数据由不同文档组成, 例如不同网页或源代码文件, 常见做法仍然是把它们全部连接为单个 token 序列, 并在文档之间加入 `<|endoftext|>` 等分隔符。

数据加载器把这个序列转换成 batch 流。每个 batch 包含 B 个长度为 m 的序列, 以及它们各自同样长度为 m 的下一个 token 目标。例如, 当 $B = 1, m = 3$ 时, $([x_2, x_3, x_4], [x_3, x_4, x_5])$ 就是一个可能的 batch。

这种数据加载方式从多个方面简化了训练。首先, 任何 $1 \leq i \leq n - m$ 都能给出一个有效训练序列, 因此抽样训练序列非常简单。其次, 所有训练序列长度相同, 无需填充输入序列; 这提高了硬件利用率, 也方便增大 batch size B 。最后, 抽样训练数据不需要加载完整数据集, 因此也能轻松处理无法放入内存的大型数据集。

问题 (data_loading): 实现数据加载 (2 分)

提交内容: 编写一个函数, 接收 NumPy 数组 x (包含 token ID 的整数数组)、`batch_size`、`context_length` 以及 PyTorch 设备字符串 (例如 `'cpu'` 或 `'cuda:0'`), 返回一对 Tensor: 抽样得到的输入序列和对应的下一个 token 目标。

两个 Tensor 的 shape 都应为 $(\text{batch_size}, \text{context_length})$, 内容为 token ID, 并且都应放在所请

求的设备上。

要使用课程组提供的测试检查实现，首先完成测试适配器 `adapters.run_get_batch`，然后运行：

```
uv run pytest -k test_get_batch
```

低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果准备在 CPU 或 Apple Silicon 上训练语言模型，需要把数据移动到正确设备；稍后模型也应使用同一设备。

使用 CPU 时，设备字符串可以设为 `'cpu'`；使用 Apple Silicon（M 系列芯片）时，可以设为 `'mps'`。

MPS 相关资料：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集太大，无法装入内存怎么办？可以使用名为 `mmap` 的 Unix 系统调用。它会把磁盘文件映射到虚拟内存，并在访问相应内存位置时惰性加载文件内容。因此，可以“假装”整个数据集都在内存中。

NumPy 通过 `np.memmap` 实现这一功能。如果数组最初使用 `np.save` 保存，也可以通过 `np.load` 的 `mmap_mode='r'` 参数实现。它们会返回类似 NumPy 数组的对象，在访问条目时按需加载。

训练期间从数据集，也就是 NumPy 数组中抽样时，务必用内存映射模式加载数据集：具体选择 `np.memmap`，还是 `np.load(..., mmap_mode='r')`，取决于数组的保存方式。还要指定与所加载数组匹配的 `dtype`。最好显式检查内存映射数据是否正确，例如确认其中没有超过预期词表大小的值。

5.2 检查点

除了加载数据，还需要在训练期间保存模型。运行训练任务时，我们经常希望恢复一项中途停止的训练，例如任务超时或机器故障。即使训练顺利完成，之后也可能需要访问中间模型，例如事后研究训练动态，或从训练不同阶段的模型中采样。

检查点应保存恢复训练所需的全部状态。最基本的是恢复模型权重。如果使用 AdamW 等有状态优化器，还需要保存优化器状态，例如 AdamW 的矩估计。最后，为了恢复学习率调度，还必须知道训练停止时的迭代次数。

PyTorch 可以方便地保存这些内容：每个 `nn.Module` 都有 `state_dict()` 方法，返回包含全部可学习权重的字典；之后可以使用配套方法 `load_state_dict()` 恢复这些权重。任何 `torch.optim.Optimizer` 也提供同样的接口。

最后，`torch.save(obj, dest)` 可以把对象写入文件路径或类文件对象。这个对象可以是字典，其中既可包含 `Tensor`，也可包含整数等普通 Python 对象。之后可使用 `torch.load(src)` 将其重新载入内存。

问题 (checkpointing)：实现模型检查点 (1 分)

实现以下两个函数来保存和加载检查点。

```
def save_checkpoint(model, optimizer, iteration, out)
```

该函数应把模型、优化器与迭代次数的全部状态写入类文件对象 `out`。可以使用模型和优化器的 `state_dict` 方法获取相关状态，再用 `torch.save(obj, out)` 把 `obj` 写入 `out`；这里 PyTorch 既支持路径，也支持类文件对象。典型做法是让 `obj` 为字典，但只要之后能够加载检查点，可以使用任意格式。

参数如下：

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`
- `iteration: int`
- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

```
def load_checkpoint(src, model, optimizer)
```

该函数应从 `src`（路径或类文件对象）加载检查点，并恢复其中的模型与优化器状态；最后返回检查点中保存的迭代次数。可以用 `torch.load(src)` 取回 `save_checkpoint` 保存的内容，再用模型和优化器的 `load_state_dict` 恢复之前的状态。

参数如下：

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

实现适配器 `adapters.run_save_checkpoint` 与 `adapters.run_load_checkpoint`，并确保以下命令通过：

```
uv run pytest -k test_checkpointing
```

5.3 训练循环

终于可以在主训练脚本中组合你已经实现的全部组件。最好让使用不同超参数启动训练变得简单，例如把超参数作为命令行参数，因为稍后你需要多次运行实验，研究不同选择对训练的影响。

问题 (training_together)：组合全部组件 (4 分)

提交内容：编写一个训练循环脚本，在用户提供的输入上训练模型。具体而言，建议训练脚本至少支持：

- 配置并控制各种模型与优化器超参数；

- 使用 `np.memmap` 以内存高效的方式加载大型训练集和验证集；
- 把检查点序列化到用户提供的路径；
- 定期记录训练和验证性能，例如输出到控制台和/或 Weights & Biases 等外部服务。⁸

6 生成文本

现在已经可以训练模型，最后还需要让模型生成文本。

回顾一下：语言模型接收一个长度为 `sequence_length` 的整数序列，可能包含 `batch` 维度，并产生大小为 `(sequence_length, vocab_size)` 的矩阵。序列中每个元素对应一个概率分布，用于预测该位置之后的下一个 `token`。下面编写一些函数，把这些预测转换为新序列的采样方案。

Softmax

按照标准惯例，语言模型输出是最后一个线性层的输出，也就是 logits。因此，需要使用前面公式 (10) 中的 softmax 操作，把它转换成归一化概率。

解码

要让模型生成，也就是解码文本，我们向模型提供一个前缀 `token` 序列，即 “prompt”，再让它产生词表上的概率分布，预测序列中的下一个 `token`。然后从这个词表分布中采样，确定下一个输出 `token`。

具体而言，解码过程的一步接收序列 $x_{1\dots t}$ ，并按照下式返回 `token` x_{t+1} ：

$$P(x_{t+1} = i \mid x_{1\dots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)}, \quad (21)$$

其中：

$$v = \text{TransformerLM}(x_{1\dots t})_t \in \mathbb{R}^{\text{vocab_size}}. \quad (22)$$

`TransformerLM` 接收长度为 `sequence_length` 的序列，生成大小为 `(sequence_length, vocab_size)` 的矩阵。我们想得到位置 t 的下一个 `token` 预测，因此取该矩阵最后一个元素。

反复从这些单步条件分布中采样，并把上一步生成的输出 `token` 追加到下一解码时间步的输入中，直到生成序列结束 `token <|endoftext|>`，或者达到用户指定的最大生成 `token` 数，就得到一个基本解码器。

解码技巧

本作业实验使用小型模型，而小型模型有时会生成质量很低的文本。两个简单的解码技巧可以缓解这一问题。

第一种是温度缩放（temperature scaling）：用温度参数 τ 修改 softmax，新的 softmax 为：

⁸<https://wandb.ai>

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)}. \quad (23)$$

注意, 当 $\tau \rightarrow 0$ 时, v 的最大元素会占据主导, softmax 输出会变成集中在该最大元素上的 one-hot 向量。

第二种技巧是 nucleus sampling, 也称 top- p 采样: 通过截断低概率 token 来修改采样分布。设 q 是对大小为 vocab_size 的 logits 应用可能经过温度缩放的 softmax 后得到的概率分布。超参数为 p 的 nucleus sampling 按下式产生下一个 token:

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j}, & i \in V(p), \\ 0, & \text{otherwise,} \end{cases} \quad (24)$$

其中, $V(p)$ 是满足 $\sum_{j \in V(p)} q_j \geq p$ 的最小索引集合。可以先按概率大小对分布 q 排序, 再从最大的词表元素开始选择, 直到累积概率达到目标 p , 从而方便地计算该集合。

问题 (decoding): 解码 (3 分)

提交内容: 实现一个从语言模型解码的函数。建议支持以下功能:

- 为用户提供的 prompt 生成补全, 即接收某个 $x_{1..t}$ 并持续采样, 直到遇到 `<|endoftext|>` token;
- 允许用户控制最大生成 token 数;
- 给定所需温度值, 在采样前对预测的下一个 token 分布应用 softmax 温度缩放;
- 给定用户指定的阈值, 执行 top- p 采样 [A. Holtzman et al., 2020], 也称 nucleus sampling。

7 实验

现在可以组合所有内容, 在预训练数据集上训练小型语言模型。

7.1 如何运行实验与提交内容

理解 Transformer 各架构组件背后理由的最佳方法, 是亲自修改并运行模型。动手经验无可替代。

为此, 必须能够快速、一致地开展实验, 并记录自己的操作。为了快速实验, 我们将在一个小型模型 (总参数约 17M) 和简单数据集 TinyStories 上运行大量实验。为了保持一致, 你需要系统地消融各个组件并改变超参数; 为了保留记录, 本节要求提交实验日志以及每项实验对应的学习曲线。

为了能够提交损失曲线, 务必定期评估验证损失, 并同时记录步数和实际经过时间。Weights & Biases 等日志基础设施可能会有所帮助。

问题 (experiment_log): 实验日志 (3 分)

为训练与评估代码建立实验追踪基础设施，使其能够相对于梯度步数和实际经过时间追踪实验及损失曲线。

提交内容：实验日志基础设施代码，以及本节后续作业问题的实验日志，也就是记录你尝试过的全部内容的文档。

7.2 TinyStories

首先使用一个非常简单的数据集 TinyStories [R. Eldan et al., 1]。模型在这个数据集上训练很快，而且可以观察到一些有趣行为。数据集获取说明见第 1 节。下面是一条数据示例。

示例 (`tinystories_example`)：TinyStories 中的一个样本

从前，有一个名叫 Ben 的小男孩。Ben 喜欢探索身边的世界。他见过许多令人惊叹的东西，例如商店里陈列的漂亮花瓶。有一天，Ben 走过商店时，发现了一个非常特别的花瓶。看到它时，Ben 惊叹不已！他说：“哇，这个花瓶真漂亮！我可以买下它吗？”店主微笑着说：“当然可以。你可以把它带回家，让所有朋友看看它有多棒！”于是 Ben 把花瓶带回了家，他感到无比自豪！他把朋友们叫来，给他们展示这个漂亮的花瓶。朋友们都觉得花瓶很美，也不敢相信 Ben 竟然如此幸运。这就是 Ben 在商店里发现漂亮花瓶的故事！

7.2.1 超参数调整

我们先给出一些非常基本的起始超参数，再由你寻找其他表现良好的设定。

- **词表大小：10,000。**常见词表大小从数万到数十万不等。应改变这一参数，观察词表与模型行为如何变化。
- **上下文长度：256。**TinyStories 等简单数据集可能不需要很长的序列，但之后的 OpenWebText 数据可能需要改变这一设置。尝试不同取值，观察它对每次迭代运行时间和最终困惑度的影响。
- **`d_model`：512。**这略小于许多小型 Transformer 论文使用的 768 维，但可以加快运行。
- **`d_ff`：1,344。**这大约是 $\frac{8}{3}d_{model}$ ，同时也是 64 的倍数，有利于 GPU 性能。
- **RoPE 的 θ 参数 Θ ：10,000。**
- **层数与注意力头数：4 层、16 个头。**二者结合会得到约 17M 个不计嵌入层的参数，属于相当小的 Transformer。
- **处理的 token 总数：327,680,000。**`batch size * 总步数 * 上下文长度` 应约等于该值。

还需要通过反复试验，为下列其他超参数找到良好的默认值：学习率、学习率预热、AdamW 的其他超参数 (β_1 、 β_2 、 ϵ) 以及权重衰减。D. P. Kingma 等人 [22] 给出了一些典型选择。

7.2.2 组合全部内容

现在可以把所有内容组合起来：取得训练好的 BPE 分词器，对训练数据集进行分词，再把数据送入你编写的训练循环。

重要说明：如果实现正确且高效，使用上述超参数在一张 B200 GPU 上的运行时间应约为 20 到 30 分钟。如果运行时间长得多，请检查数据加载、检查点保存或验证损失代码是否成为瓶颈，并确认实现进行了正确的批处理。

7.2.3 调试模型架构的提示与技巧

强烈建议熟悉 IDE 内置调试器，例如 VSCode 或 Zed；与使用 `print` 语句调试相比，它能节省很多时间。如果使用文本编辑器，可以使用 `ipdb` 等工具。调试模型架构时还可以采用以下良好实践：

- 开发任何神经网络架构时，一个常见的第一步是在单个 minibatch 上过拟合。如果实现正确，应能迅速把训练损失降到接近零。
- 在模型的不同组件中设置调试断点，检查中间 Tensor 的 shape，确认它们符合预期。
- 监控激活值、模型权重与梯度的范数，确认它们没有爆炸或消失。

问题 (`learning_rate`)：调整学习率 (2 B200 小时, 3 分)

学习率是最重要的待调超参数之一。以训练得到的基础模型为起点，回答以下问题：

1. 对多个学习率执行超参数扫描，并报告最终损失；如果优化器发散，则注明这一点。**提交内容：**多个学习率对应的学习曲线，并说明你的超参数搜索策略。

提交内容：一个在 TinyStories 上逐 token 验证损失不高于 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练少量步骤

如果使用 `cpu` 或 `mps`，应把处理的 token 总数减少到 40,000,000；这已经足以生成相当流畅的文本。也可以把目标验证损失从 1.45 放宽到 2.00。

在配备 36 GB 内存的 M4 Max 上运行课程组参考实现并使用调好的学习率时，我们采用：
 $\text{batch size} * \text{总步数} * \text{上下文长度} = 32 * 5000 * 256 = 40,960,000$ token。在 CPU 上需要 1 小时 22 分钟，在 MPS 上需要 36 分钟。到第 5,000 步时，验证损失为 1.80。

其他提示：

- 使用 N 个训练步骤时，建议调整余弦学习率衰减调度，使它恰好在第 N 步完成衰减，也就是达到最小学习率。
- 使用 MPS 时，不要使用 TF32 kernel，也就是说，不要像使用 CUDA 设备时那样设置 `torch.set_float32_matmul_precision('high')`。课程组曾在 MPS (PyTorch 2.9.0) 上启用 TF32 kernel，发现后端有时会悄无声息地采用损坏的 kernel，导致训练不稳定。
- 可以使用 `torch.compile` 对模型进行 JIT 编译，从而加快训练。具体来说：

- 在 CPU 上, 使用 `model = torch.compile(model)` 编译模型。
 - 在 MPS 上, 可以使用 `model = torch.compile(model, backend="aot_eager")` 对反向传播做一定优化。截至 PyTorch 2.9.0, MPS 尚不支持使用 Inductor 编译。
2. 经验上常说, 最佳学习率位于“稳定性边缘”(edge of stability)。研究学习率开始发散的位置与你找到的最佳学习率之间有何关系。**提交内容:** 一组学习率逐渐增大的学习曲线, 其中至少包含一次发散的运行; 并分析这与收敛速度之间的关系。

接下来改变 batch size, 观察训练会发生什么。Batch size 很重要: 更大的矩阵乘法能够让 GPU 获得更高效率, 但是否总应该使用很大的 batch size? 下面通过实验找出答案。

问题 (batch_size_experiment): 改变 batch size (1 B200 小时, 1 分)

改变 batch size, 范围从 1 一直到 GPU 内存上限。中间至少尝试几个不同值, 包括 64、128 等典型大小。

提交内容: 不同 batch size 运行对应的学习曲线; 如有必要, 应重新优化学习率。

提交内容: 用几句话讨论关于 batch size 的发现及其对训练的影响。

有了解码器之后, 现在可以生成文本, 并观察模型的表现。作为参考, 输出质量至少应达到下面示例的水平。

示例 (ts_generate_example): TinyStories 语言模型的输出样本

从前, 有一个名叫 Lily 的漂亮女孩。她喜欢吃口香糖, 尤其是那块黑色的大口香糖。有一天, Lily 的妈妈请她帮忙做晚饭。Lily 兴奋极了! 她很喜欢帮助妈妈。Lily 的妈妈为晚饭煮了一大锅汤。Lily 非常开心地说: “谢谢你, 妈妈! 我爱你。” 她帮妈妈把汤倒进一个大碗。晚饭后, Lily 的妈妈又做了一些美味的汤。Lily 很喜欢! 她说: “谢谢你, 妈妈! 这汤太好喝了!” 妈妈笑着说: “Lily, 你喜欢就好。” 她们完成了烹饪, 又继续一起做饭。故事结束。 **低资源提示: 在 CPU 或 Apple Silicon 上生成文本**

如果使用只处理 40M token 的低资源配置, 生成结果看上去应仍然像英语, 但不会像上面的示例那样流畅。例如, 下面是课程组使用在 40M token 上训练的 TinyStories 语言模型得到的样本输出:

从前, 有一个名叫 Sue 的小女孩。Sue 有一颗她非常喜欢的牙齿。那是他最好的头。有一天, Sue 出去散步, 遇到了一只瓢虫! 他们成了好朋友, 一起在小路上玩耍。

“嘿, Polly! 我们出去吧!” Tim 说。Sue 抬头望向天空, 发现很难找到一条闪亮地跳舞的路。她笑了, 同意帮助那段话!

当 Sue 看着天空移动, 那是什么。她

下面是具体问题与要求。

问题 (generate): 生成文本 (1 分)

使用解码器和训练好的检查点, 报告模型生成的文本。为了得到流畅输出, 可能需要调整温度、top- p 等解码器参数。

提交内容: 至少 256 个 token 的文本转储; 如果更早遇到第一个 `<|endoftext|>` token, 则到该 token 为止。另外, 简要评论输出的流畅程度, 并指出至少两个影响输出质量好坏的因素。

7.3 消融与架构修改

理解 Transformer 的最佳方法是亲自修改它, 并观察其行为。下面进行几个简单的消融和修改实验。

消融 1: 层归一化

人们经常说, 层归一化对 Transformer 的训练稳定性非常重要。但或许我们想冒点险。移除每个 Transformer block 中的 RMSNorm, 看看会发生什么。

问题 (layer_norm_ablation): 移除 RMSNorm 并训练 (0.5 B200 小时, 1 分)

从 Transformer 中移除全部 RMSNorm 后进行训练。使用之前的最佳学习率时会发生什么? 改用较低学习率能否恢复稳定性?

提交内容: 移除 RMSNorm 后训练的学习曲线, 以及最佳学习率对应的学习曲线。

提交内容: 用几句话评论 RMSNorm 的影响。

接下来研究另一项乍看之下有些任意的层归一化选择。Pre-norm Transformer block 定义为:

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)), \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构少数几项已经形成“共识”的修改之一。原始 Transformer 使用 post-norm:

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)), \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

下面恢复为 post-norm, 观察会发生什么。

问题 (pre_norm_ablation): 实现 post-norm 并训练 (0.5 B200 小时, 1 分)

把 pre-norm Transformer 实现改成 post-norm, 使用 post-norm 模型训练并观察结果。

提交内容: post-norm Transformer 的学习曲线, 并与 pre-norm Transformer 对比。

由此可以看到, 层归一化对 Transformer 的行为有重大影响, 甚至层归一化所在的位置也很重要。

消融 2: 位置嵌入

接下来研究位置嵌入对模型性能的影响。具体而言, 我们会把使用 RoPE 的基础模型, 与完全不加入位置嵌入的 NoPE 进行比较。

事实证明, 本作业实现的这种带因果掩码的仅解码器 Transformer, 理论上可以在不显式提供位置嵌入的情况下推断相对或绝对位置信息 [Y.-H. H. Tsai et al., 2019; A. Kazemnejad et al., 2023]。下面通过实验比较 NoPE 与 RoPE 的表现。

问题 (no_pos_emb): 实现 NoPE (0.5 B200 小时, 1 分)

修改带 RoPE 的 Transformer 实现, 彻底移除位置嵌入信息, 并观察结果。

提交内容: 比较 RoPE 与 NoPE 性能的学习曲线。

消融 3: SwiGLU 与 SiLU

接下来遵循 N. Shazeer [20], 比较 SwiGLU 前馈网络与只使用 SiLU 激活、但不使用门控线性单元 (GLU) 的前馈网络, 以检验门控在前馈网络中的重要性:

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

回顾一下: 在 SwiGLU 实现中, 内部前馈层维度约设为 $d_{ff} = \frac{8}{3}d_{\text{model}}$, 同时保证 $d_{ff} \bmod 64 = 0$, 以利用 GPU tensor core。在本消融基线中, FFN_{SiLU} 应改为设置 $d_{ff} = 4d_{\text{model}}$, 从而与默认 SwiGLU 前馈网络大致匹配参数量; SwiGLU 有三个权重矩阵, 而这里有两个。

问题 (swiglu_ablation): SwiGLU 与 SiLU (0.5 B200 小时, 1 分)

提交内容: 在参数量大致匹配的条件下, 比较 SwiGLU 与 SiLU 前馈网络性能的学习曲线。

提交内容: 用几句话讨论你的发现。

低资源提示: GPU 资源有限的在线学习者应在 TinyStories 上测试修改

作业剩余部分将转向规模更大、噪声更多的网络数据集 OpenWebText, 并尝试架构修改, 以及选择性地向课程排行榜提交结果。

在 OpenWebText 上把语言模型训练到能够流畅生成需要很长时间。因此, 建议 GPU 资源有限的在线学习者继续在 TinyStories 上测试修改, 并以验证损失作为性能评估指标。

7.4 在 OpenWebText 上运行

现在转向由网络爬取数据构建的、更标准的预训练数据集。课程组还提供了单个文本文件形式的 OpenWebText [A. Gokaslan et al., 2019] 小样本；访问方式见第 1 节。

下面是一条 OpenWebText 示例。注意，它的文本更加真实、复杂且多样。建议浏览训练数据集，了解网络抓取语料库中的训练数据是什么样子。

示例 (owt_example)：OWT 中的一个样本

Baseball Prospectus 的技术总监 Harry Pavlidis 在聘用 Jonathan Judge 时冒了一个险。

Pavlidis 知道，正如 Alan Schwarz 在《The Numbers Game》中所写：“美国文化中，没有哪个角落比棒球运动员的表现得到更精确的计数、更热情的量化。”只需随手点击几下，你就可以知道 Noah Syndergaard 的快速球在飞向本垒的途中每分钟旋转超过 2,100 次；Nelson Cruz 在 2016 年符合统计资格的击球手中拥有全联盟最高的平均击球初速；以及无数其他仿佛取自电子游戏或科幻小说的细节。不断上涨的数据海洋，让棒球文化中一个愈发重要的角色获得了力量：业余数据分析者。

这种赋权也带来了更多审视，既针对测量结果，也针对这些结果背后的人与出版机构。在 Baseball Prospectus 工作的 Pavlidis 非常了解定量分析不够完善时随之而来的批评。他也知道，网站对接球表现的衡量指标需要重新设计，而且必须由一位学识渊博、能够处理复杂统计建模问题的人来完成这项工作。

“他把我们吓了一跳。” Harry Pavlidis

根据 Judge 的文章，以及两人在网站赞助的球场活动中的交流，Pavlidis 直觉认为 Judge “懂这个”。……

注意：本实验可能需要重新调整学习率或 batch size 等超参数。

问题 (main_experiment)：在 OWT 上实验 (2 B200 小时，2 分)

使用与 TinyStories 相同的模型架构和训练迭代总数，在 OpenWebText 上训练语言模型。这个模型表现如何？

提交内容：语言模型在 OpenWebText 上的学习曲线。说明损失与 TinyStories 相比有何不同，以及如何解释这些损失。

提交内容：OpenWebText 语言模型生成的文本，格式与 TinyStories 输出相同。文本流畅度如何？尽管模型与计算预算和 TinyStories 相同，为什么输出质量更差？

7.5 自选修改与排行榜

恭喜你完成到这里，作业即将结束。现在尝试改进 Transformer 架构，并比较自己的超参数与架构和班上其他同学相比表现如何。

排行榜规则

除以下规则外，没有其他限制：

- **运行时间**：提交程序在 B200 上最多运行 45 分钟。如果使用 SLURM 或 Modal，可以考虑在提交脚本中强制执行这一限制。
- **数据**：只能使用课程组提供的 OpenWebText 训练数据集。

除此之外，可以自由尝试任何方案。

如果需要实现思路，可以参考：

- 当前先进的开源大语言模型系列，例如 Llama 3 [A. Grattafiori et al., 2024] 或 Qwen 2.5 [A. Yang et al., 2024]。
- NanoGPT speedrun 仓库：<https://github.com/KellerJordan/modded-nanogpt>。社区成员在其中发布了许多“小规模语言模型预训练竞速”的有趣修改。例如，把输入与输出嵌入的权重绑定在一起，是一种可以追溯到原始 Transformer 论文的常见修改；参见 A. Vaswani 等人 [8] 第 3.4 节和 A. Chowdhery 等人 [16] 第 2 节。如果尝试权重绑定，可能需要减小嵌入层或 LM head 初始化的标准差。

正式开始完整的 45 分钟运行前，应先在 OpenWebText 的小型子集或 TinyStories 上测试这些修改。

需要提醒的是，在这个排行榜上表现良好的某些修改，未必能推广到更大规模的预训练。课程的缩放定律单元会进一步探讨这一点。

问题 (leaderboard)：排行榜 (10 B200 小时, 6 分)

按照上述排行榜规则训练模型，目标是在 0.75 B200 小时内尽可能降低语言模型的验证损失。

提交内容：最终记录的验证损失；与之对应的学习曲线，其中横轴必须清楚标示实际经过时间，并且小于 45 分钟；以及对所做工作的说明。排行榜提交至少应超过验证损失为 5.0 的朴素基线。

提交地址：<https://github.com/stanford-cs336/assignment1-basics-leaderboard>

参考文献

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019.
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.
- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [5] P. Gage, “A new algorithm for data compression,” *C Users Journal*, vol. 12, no. 2, pp. 23-38, Feb. 1994.
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Un-

supervised Multitask Learners.” 2019.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.

[8] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.

[9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSWLT*, 2019.

[10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.

[11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.

[12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.

[13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.

[14] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: <https://arxiv.org/abs/2407.21783>

[15] A. Yang et al., “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.

[16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.

[17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016.

[18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1702.03118>

[19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1612.08083>

[20] N. Shazeer, “GLU Variants Improve Transformer.” 2020.

[21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.

[22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.

[23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.

[24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.

[25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020.

[26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.

[27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.

[28] Y.-H. H. Tsai, S. Bai, M. Yamada, J.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’s Attention via the Lens of Kernel,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference*

on Natural Language Processing (EMNLP-IJCNLP), K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4344-4353. doi: 10.18653/v1/D19-1443.

[29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: <https://openreview.net/forum?id=Drrl2gcjzl>